

华北电力大学

过程作业 1 报告

(2025 -- 2026 年度第 二 学期)

名 称: 作业 1

题 目: 模拟空战单机版

院 系: 计算机系

班 级:

学 号:

学生姓名:

指导教师: 王晓辉

成 绩:

日期: 2026 年 5 月 21 日

摘要

目标: 按 View → Controller → Model 分层与 **接口先行** 原则, 实现**顶视海空战场**下单机可玩的空战仿真, 界面实时显示 **FPS** 与**红蓝剩余架数**, 支持导弹交战与雷达探测表现。

实现要点: 浏览器内以前端 TypeScript 推算战场状态; LocalGameController 基于 ****requestAnimationFrame**** 驱动主循环; 核心契约定义于 interfaces.ts (IFighter、IMissile、IRadarSystem、ICombatJudger、IAIController、INetworkProtocol) ; 渲染采用 Canvas, 并结合 **离屏缓冲**、**脏矩形**、**Path2D 缓存** 等控制绘制开销; 碰撞侧采用 **空间网格粗筛** 与导弹 **对象池**。

运行方式: 开发态 npm run dev; 交付静态资源为 air-combat-frontend/dist/, 需经 HTTP 访问 (步骤见 ****README_ 单机版.md §2**** 与本文 **§2**)。

胜负判定: 某方战机全部被击落。

1. 实验目的

1.1 掌握 Model / View / Controller 分层与层间依赖约束; 实践 **接口先行**; 以 **FPS** 为核心做性能分析与优化描述; 理解 **计算与显示分离** 及 **线程池** 在多线程架构中的作用 (本实现的线程模型见 **§6**)。

1.2 在国防空战与体系作战题材下完成仿真: 体会预警探测、火力与指挥分层协作与软件分层、接口解耦的对应关系。

1.3 View 只负责展示快照与捕获输入, 战场逻辑集中在 Model, Controller 持有主循环并协调步进与刷新。

1.4 完成顶视空战单机闭环的主体功能, 并形成可核查的架构、性能、网络协议预留与测试记录。

2. 实验环境与运行说明

项目	内容
操作系统	Windows / macOS
浏览器	Chrome / Edge 等主流浏览器
运行依赖	Node.js 18+、npm; 可选 Python 3 仅用于托管已构建的 dist/

2.1 最短复现命令:

```
cd air-combat-frontend
npm install
npm run dev
```

在浏览器打开终端提示的本地地址 `http://localhost:5173`, 首页选择 **「进入单机模式」**。

2.2 使用 Python 托管 dist/

前提: 本机已安装 **Python 3** (构建仍依赖上文的 npm run build; Python **不能代替** Node 完成打包, 只负责把已生成的 dist/ 当作网站根目录发布)。若未安装 Python, 请仅用 npm run preview 或安装 Python 3 后再用下列命令。

2.2.1 进入构建产物目录 (目录内应有 index.html 与 assets/):

```
cd air-combat-frontend\dist
```

2.2.2 启动内置静态服务。下例端口为 8080 (若被占用可改为 8090 等, 地址中端口需一致):

```
python -m http.server 8080
```

2.2.3 浏览器访问 ****http://127.0.0.1:8080/**** (端口与命令一致), 首页选择 **「进入单机模式」**。 **注意事项:**

- **工作目录必须是 dist:** 若在 air-combat-frontend 或仓库根目录启动服务, index.html 不在站点根上, 易出现白屏或资源 404。
- **禁止仅用 file:// 双击 index.html**

- **端口被占用**: 终端若提示 Address already in use, 请换一个端口 (如 8090), 浏览器地址改为 http://127.0.0.1:8090/。
- **本机访问**: 127.0.0.1 与 localhost 通常等价; 若仅 localhost 异常可改试 127.0.0.1。
- **关闭服务**: 在运行 http.server 的终端按 Ctrl+C。

运行入口界面

以下为本地开发与进入单机模式的入口界面示意。



图 1 运行入口主页 (开发: Vite Dev; 可先选「单机模式」入口)

3. 需求与实现对照

3.1 核心功能

编号	需求要点	本实现说明
1 战场	海空域俯视地图; 敌机从边缘侵入; 预警机开视野、战机视野可简化	Canvas 顶视战场; 雷达/探测数据用于「开视野」式目标标注 (见 GameRenderer、RadarSystem)
2 操控	键盘 WASD/方向键与鼠标双模; 鼠标航向; 左键导弹;	InputController 汇聚键鼠; 左键/J/空格导弹, 右键与 K 为炸弹
3 导弹	空空导弹、简易追踪; 命中后敌机消失	Missile 追踪逻辑 + 碰撞判定后敌机 alive=false/退场
4 雷达	空警-500 类预警探测范围、地图显示敌位置;	预警机与普通机雷达范围差异;
5 胜负	敌机全部击落胜利; 己方全灭失败; 实时剩余架数	按击落判定胜利 ;
6 FPS	界面实时帧率为性能核心指标	HUD 显示 fpsCounter
7 界面信息	地图、敌我位置航向、雷达、FPS、剩余架数、当前战机状态等	HUD + BattleSnapshot; 雷达圈/标记见渲染层

3.2 架构与接口

编号	约束或要求	本实现说明
A 分层	View / Controller / Model; View 不直接 new/调用 Model; Model 不依赖 UI	Vue 组件只拿快照; Battlefield 等在 src/game, 无 Vue 引用; 步经由 LocalGameController
B 接口	先定义 IFighter、IMissile、IRadarSystem、ICombatJuder、IAIController、INetworkProtocol	src/game/interfaces.ts (TypeScript 驼峰命名, 与常见的 Move/Fire 能力一一对应)
C 网络	协议预留, 通信可不实装	见 §7

4. 系统架构 (分层与组件)

4.1 分层结构

分层	职责	代表文件 (相对 air-combat-frontend/)
View	页面布局、画布挂载、HUD、输入事件绑定	src/components/BattleCanvas.vue、GameHud.vue
Controller	游戏主循环、输入汇聚、驱动 Model 步进、向 View 提供只读快照	src/controllers/LocalGameController.ts、InputController.ts
Model	战机/导弹/AI/雷达/碰撞/胜负	src/game/Battlefield.ts、Fighter.ts、CombatJudger.ts 等

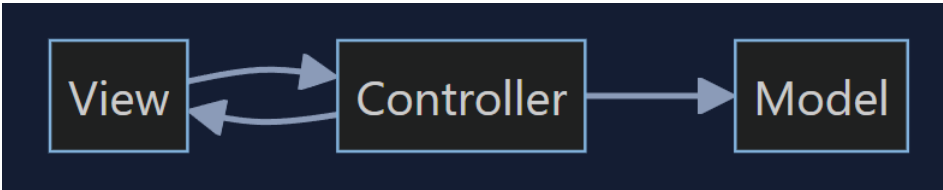


图 2 分层 (MVC) 架构示意

4.2 组件交互与数据流 (文字)

4.2.1 输入流: 键盘、鼠标在 View 中监听, 经 InputController 归一后交给 LocalGameController。

4.2.2 状态流: 每帧 (或固定时间步) LocalGameController 调用 Battlefield 更新, 生成 ****BattleSnapshot**** 类快照; View 仅读取快照绘制, 不直接改写 Model 内对象。

4.2.3 设计意图: 满足「计算与显示分离」与模型层 不依赖 Vue/DOM 的约束, 便于单元理解与联机扩展时替换同步源。

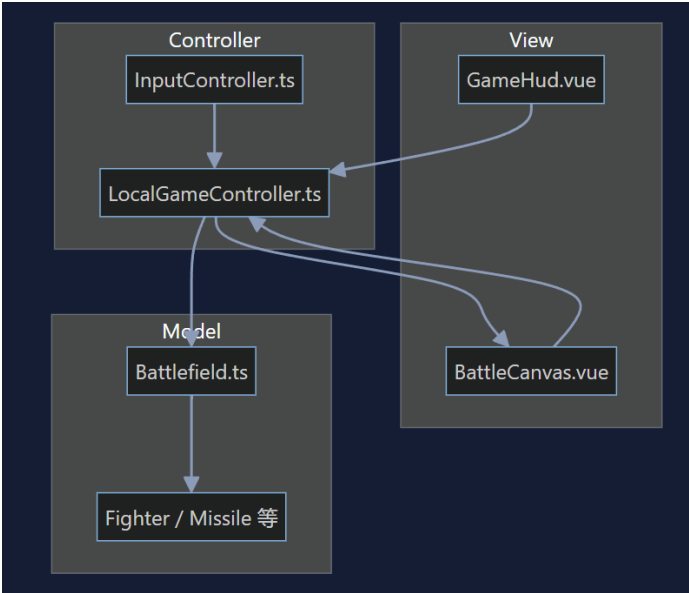


图 3 数据流示意

5. 接口先行与主要类型

核心接口定义于 src/game/interfaces.ts, 实体类 implements 对应接口。

接口	责任概述
IFighter	战机移动、武器发射、伤害、步进与状态导出
IMissile	导弹发射、追踪、爆炸/失效、步进与状态导出
IRadarSystem	扫描、探测目标列表、作用距离
ICombatJudger	碰撞粗筛/判定、战果相关逻辑
IAIController	敌方 AI 决策与执行
INetworkProtocol	战场消息的序列化 / 反序列化抽象

消息枚举与命名见 `src/game/types.ts`; JSON 实现见 `src/game/JsonNetworkProtocol.ts`。单机运行不建立对战 WebSocket; 接口与消息类型与联机路径共用同一套抽象, 便于日后扩展。

6. 性能优化分析

已采用的优化手段

优化领域	常见手段	本实现
碰撞检测	四叉树 / 网格 、AABB 粗测 + 精测	空间网格粗筛 + 距离等精判 (CombatJudger / Battlefield)
寻路/追踪	2D 简化、对象池等	导弹 对象池 ; 追踪在 Model 内简化实现
界面刷新	双缓冲 、 脏矩形 、计算与渲染分离	GameRenderer 离屏缓冲 、HUD 等 脏矩形/签名 ; 步进在 Controller、绘制在 View
线程池	后台并行碰撞/AI 等	见 §6
对象池	导弹/特效复用	导弹从池中取用与回收

7. 网络对战接口预留

按课设要求: **预留**联机消息协议即可, **不要求**在本仓库单机 zip 内完成可联机对战。须定义 INetworkProtocol、核心消息类型, 并说明控制层如何收发 (可与全栈工程中的 OnlineGameController 一致)。

7.1 INetworkProtocol 与序列化选型

- 接口见 `src/game/interfaces.ts` (serialize、deserialize、getMessageType 等)。
- 实现类: `**JsonNetworkProtocol (src/game/JsonNetworkProtocol.ts)`, **载荷为** JSON 字符串 `**`。
- 选型说明**: JSON 可读、易调试、与浏览器 WebSocket 文本帧一致; 若需带宽与体积可再换二进制协议。

7.2 核心消息类型

`src/game/types.ts` 中 MessageType 包含:

类型	含义
MSG_FIGHTER_MOVE	战机位置/状态同步
MSG_MISSILE_LAUNCH	导弹发射通知
MSG_HIT_RESULT	命中结果通知
MSG_BATTLE_STATUS	战场态势同步

另含 MSG_PLAYER_INPUT 等扩展, 服务于工程内房间/联机流程。

7.3 控制层收发路径

- 联机**: `src/controllers/OnlineGameController.ts` 中 `socket.send(this.networkProtocol.serialize(...))` 与 `deserialize(event.data)` (需后端配合; **本单机 zip 无后端**时联机入口无法登录属正常)。
- 单机**: LocalGameController 只做本地步进, **不打开** WebSocket。

7.4 若改为全面联机（简要）

服务端维护权威战场状态，周期性推送 MSG_BATTLE_STATUS；客户端上报 MSG_FIGHTER_MOVE、MSG_MISSILE_LAUNCH；命中由服务端判定后下发 MSG_HIT_RESULT。显示层只读快照，保持分层约束。

8. 运行演示截图

战斗过程与战果界面如下（与 §3 中战场、导弹、胜负等要点对应）。

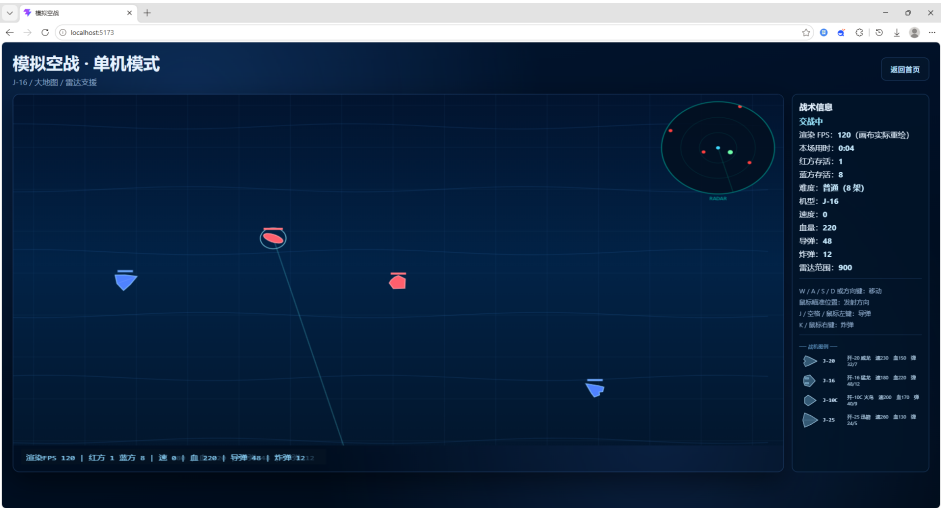


图 4 战斗过程截图 (1): 顶视战场与 HUD

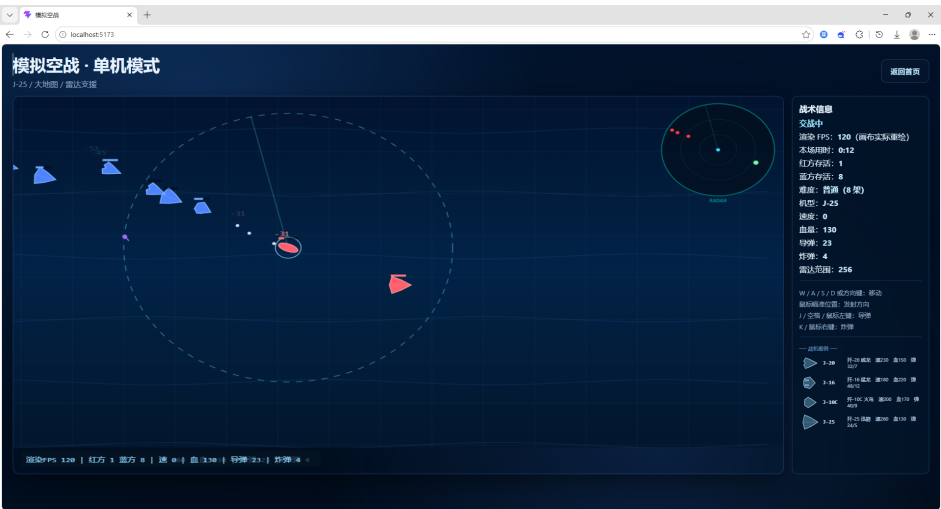


图 5 战斗过程截图 (2)

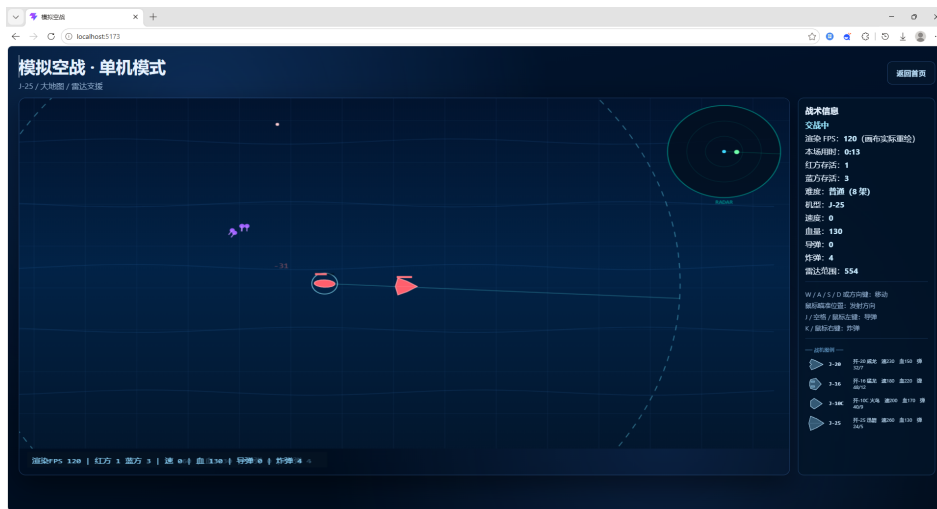


图 6 战斗过程截图 (3)

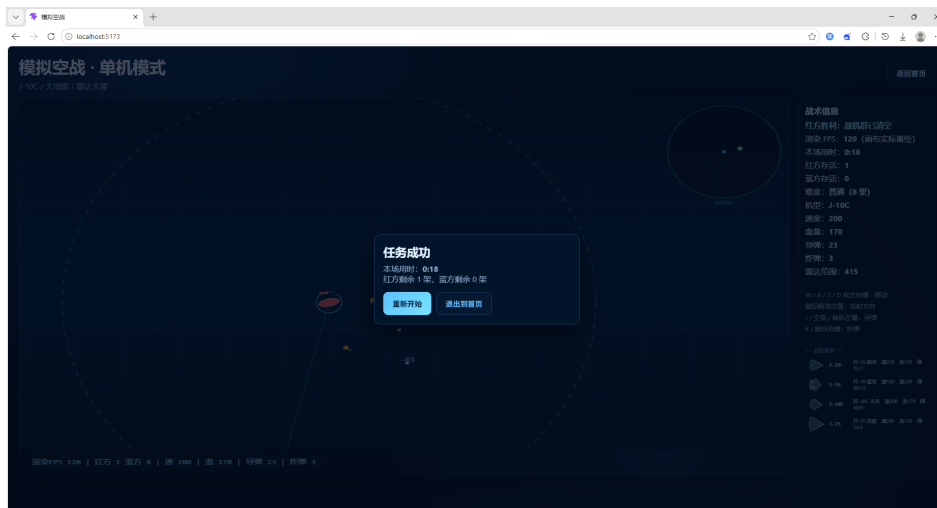


图 7 击落全部敌机后的胜利界面

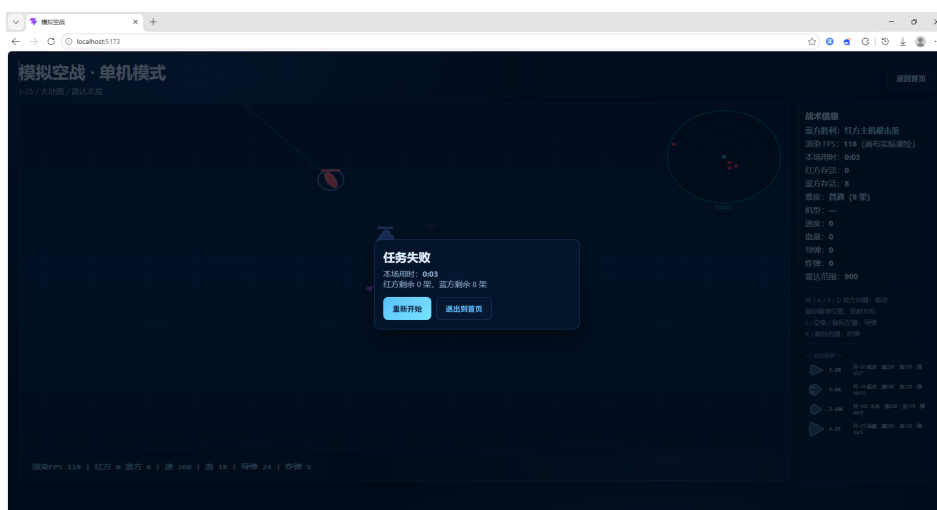


图 8 己方战机被击毁时的失败界面

9. 实验总结

9.1 完成情况

本实验在单机路径上实现了任务书所要求的 **顶视海空战场、红方玩家战斗机与预警探测、多架蓝方 AI 敌机、空空导弹与简易导引、雷达/AWACS 范围的敌情显示、胜负判定与剩余架数** 等核心功能，并在 HUD 中提供 **实时 FPS**，便于对流畅度进行定性观察。设计上坚持 **View → Controller → Model** 分层：**Vue 组件与 Canvas 仅负责展示与输入采集**；**LocalGameController** 以 `requestAnimationFrame` 组织游戏主循环；战场、实体、雷达与判定等业务逻辑集中在 Model 侧 **TypeScript** 模块中，**不依赖 Vue 与 DOM**，便于维护与测试想象中边界。

在 **接口先行** 方面，`IFighter`、`IMissile`、`IRadarSystem`、`ICombatJudger`、`IAIController` 及 `**INetworkProtocol**` 等均在 `interfaces.ts` 中定义，由具体类实现，既满足课设对契约化设计的强调，也为后续替换 AI、碰撞策略或接入网络层预留了扩展面。胜负判定以 **击落** 对方可计数战机为依据。

9.2 性能与实现的体会

单机前端采用 **浏览器主线程 + 游戏循环合一** 的模型，在保证 **帧率可读** 的前提下，通过 **空间网格粗筛** 减轻碰撞复杂度、导弹 **对象池** 降低分配压力、渲染侧 **离屏 Canvas** 与 **HUD/小地图等区域脏矩形** 减少无谓重绘（详见 §6）。这既是对任务书中「FPS/性能」项的回应，也使我体会到：**性能优化必须与架构分层一起看**——在 Controller 节拍稳定的前提下，Model 步进开销与 View 绘制成本都要落在单帧预算内。

关于 **线程池与并发**：本实验单机推演与渲染运行在统一的 `requestAnimationFrame` 节拍中，侧重 **帧内算法与绘制预算控制**；其分析结论与分层说明见 §6.3。**联机后端** 场景中常用多线程 `Tick` 等手段分担负载，可作为与单机路径并列的另一种架构选型来理解。

9.3 网络与其它预留

按课设要求，单机包内重点完成 **可玩闭环** 并完成 **接口与消息抽象**。§7 说明了 `JsonNetworkProtocol` 与 `MessageType`（战机移动、导弹发射、命中结果、战场态势等），以及单机 `**LocalGameController**` 仍以本地步进为主、接口层与 `**OnlineGameController**` 同源设计，便于在相同类型定义下扩展到联机。

9.4 小结

总体而言，本次实验加深了对 **分层架构、接口抽象、人机交互与性能折中** 的理解；在单机可演示、文档与 README 可追溯的前提下，形成了结构清晰、可运行、可追溯的实验成果。

附录：AI 辅助开发说明（诚信声明）

本项目 **源代码与文档的初稿主要借助 Cursor IDE 中的 GPT 类大模型辅助生成**。本人独立完成的工作包括：**整体与分层架构设计**、模块边界与接口约定；结合课程任务书对功能与文字的 **修改与取舍**；以及在本地环境中的 **调试、验收与核对**（含单机跑通、构建与运行时间问题排查）。

本人已理解并可说明关键模块的职责与交互（例如战场步进、`LocalGameController` 主循环、渲染与快照数据流）；